

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут прикладної математики та фундаментальних наук
Кафедра прикладної математики

Курсова робота

з курсу “Робота з великими базами даних”

Виконав:

студент групи ПМ-43

Шишковський О. І.

Перевірив:

Любінський Б. Б.

Львів — 2026 р.

ВСТУП

Актуальність теми: В умовах глобалізації світової економіки авіаційні перевезення є критично важливою складовою транспортної інфраструктури. Сучасний міжнародний аеропорт — це складний технологічний комплекс, що функціонує в режимі 24/7 і генерує колосальні обсяги даних. Щодня через інформаційні системи аеропорту проходять тисячі транзакцій: бронювання квитків, реєстрація пасажирів, логістика багажу, управління повітряним рухом та технічне обслуговування суден. Ефективність роботи аеропорту безпосередньо залежить від швидкості та точності обробки цієї інформації.

Зростання пасажиропотоку, збільшення кількості авіарейсів та посилення міжнародних вимог до безпеки польотів зумовлюють необхідність переходу від традиційних систем обліку до інформаційних систем класу VLDB (Very Large Databases). Такі системи повинні бути здатні забезпечити високу продуктивність при роботі з мільйонами транзакцій та історичними архівами даних, що накопичуються роками. Проблема полягає в тому, що стандартні реляційні бази даних (OLTP), оптимізовані для оперативної обробки поточних транзакцій, часто не справляються із завданнями складного аналітичного оброблення (OLAP) та побудови стратегічної звітності на великих масивах даних.

Використання стеку технологій Microsoft SQL Server надає потужний інструментарій для вирішення завдань керування великими даними: від надійного зберігання оперативної інформації (Database Engine) до інтеграції даних з різних джерел (SSIS), багатовимірного аналізу (SSAS) та візуалізації результатів діяльності (SSRS). Розробка такої комплексної системи дозволяє автоматизувати операційну діяльність аеропорту та створити надійне підґрунтя для прийняття стратегічних управлінських рішень на основі глибокої аналітики, що робить тему курсової роботи актуальною.

Мета і задачі дослідження: Метою курсової роботи є проектування, розробка та аналіз повнофункціональної системи керування надвеликою базою даних для предметної області «Аеропорт», з реалізацією сховища даних та системи бізнес-аналітики.

Для досягнення поставленої мети необхідно вирішити наступні

задачі:

1. Провести системний аналіз предметної області, визначити основні інформаційні об'єкти, вивчити бізнес-процеси аеропорту та формалізувати вимоги до системи.
2. Обґрунтувати вибір програмних засобів реалізації проекту, проаналізувавши можливості сучасних СУБД.
3. Розробити концептуальну (ER-діаграма) та логічну моделі даних, виконати нормалізацію відносин для забезпечення цілісності даних.
4. Реалізувати фізичну модель бази даних засобами СУБД Microsoft SQL Server, включаючи створення таблиць, обмежень цілісності, тригерів та індексів.
5. Розробити стратегію та програмні скрипти для генерації великих обсягів реалістичних тестових даних (понад 500 000 записів) для перевірки продуктивності системи під навантаженням.
6. Спроекувати та реалізувати ETL-процеси для наповнення сховища даних (Data Warehouse) з використанням SSIS.
7. Побудувати OLAP-куб для проведення багатовимірного аналізу показників ефективності роботи аеропорту засобами SSAS.
8. Створити набір інтерактивних аналітичних звітів для візуалізації даних засобами SSRS.

Об'єкт дослідження: процеси обробки інформації про планування та виконання авіаційних перевезень, управління флотом та персоналом аеропорту.

Предмет дослідження: методи та засоби проектування реляційних баз даних великого обсягу, технології інтеграції даних (ETL) та інструменти багатовимірного аналізу даних (OLAP) платформи Microsoft.

Методи дослідження: У роботі використано методи системного аналізу для декомпозиції предметної області; теорію реляційних баз даних для проектування структури даних; методи імітаційного моделювання для генерації синтетичних наборів даних; методи інженерії даних для побудови сховищ та OLAP-кубів.

Наукова новизна: полягає у розробці адаптивного алгоритму генерації пов'язаних тестових даних, який враховує складну бізнес-логіку авіаперевезень (залежність місткості від моделі, графіки

роботи екіпажів, сезонність рейсів), що дозволяє проводити реалістичне тестування продуктивності VLDB-систем.

Практичне значення: Розроблена система може бути використана як прототип для автоматизації процесів обліку та аналітики в регіональних аеропортах, а також як навчальний макет для дослідження методів оптимізації запитів у великих базах даних.

РОЗДІЛ 1. СИСТЕМНИЙ АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1. Характеристика об'єкта автоматизації Предметна область «Аеропорт» характеризується високою динамічністю процесів та значною кількістю взаємопов'язаних сутностей. Аеропорт є складним логістичним вузлом, де перетинаються потоки пасажирів, вантажів та транспортних засобів. Головним виробничим процесом є організація та виконання авіарейсу, що вимагає координації дій диспетчерських служб, екіпажів, технічного персоналу та комерційних підрозділів.

Інформаційна система аеропорту повинна обробляти дані з кількох ключових підсистем:

1. **Підсистема управління флотом:** Відповідає за облік повітряних суден. Кожен літак має унікальні характеристики (модель, серійний номер, дальність польоту, конфігурація салону), які впливають на планування рейсів.
2. **Підсистема розкладу:** Формує сезонний та добовий плани польотів. Це складна задача оптимізації, яка повинна враховувати пропускну здатність терміналів та наявність вільних бортів.
3. **Підсистема кадрового обліку:** В авіації діють жорсткі нормативи щодо робочого часу пілотів та стюардес. Система повинна відстежувати кваліфікацію, допуску до типів суден та графіки відпочинку екіпажів.
4. **Операційна підсистема:** Фіксує фактичне виконання рейсів (час зльоту/посадки, затримки, скасування) та комерційні показники (кількість пасажирів, виручка).

Специфікою даної області є накопичення великих обсягів історичних даних, які є цінним активом для аналізу. Наприклад, аналіз затримок рейсів за останні 5 років може допомогти оптимізувати розклад, а аналіз завантаженості рейсів — скоригувати цінову політику.

1.2. Опис основних бізнес-процесів У рамках роботи виділено ключові бізнес-процеси, що підлягають автоматизації. Їх формалізація дозволяє коректно спроектувати структуру бази даних.

1. Планування польотів (Flight Scheduling). Цей процес є ініціюючим для всієї системи.

- Вхідні дані: Наявний парк доступних літаків, запити від

комерційного відділу на нові напрямки.

- Опис процесу: Диспетчер формує запис про новий маршрут, вказуючи аеропорт вильоту та прильоту. Система перевіряє технічну можливість виконання рейсу (чи дозволяє дальність польоту наявної моделі літака подолати відстань). Визначається плановий час вильоту та тривалість.
- Бізнес-правило: Аеропорт відправлення не може збігатися з аеропортом призначення. Один борт не може бути призначений на два рейси, що перетинаються у часі.

2. Формування екіпажу (Crew Assignment). Критично важливий процес з точки зору безпеки.

- Вхідні дані: Запланований рейс, список доступних співробітників.
- Опис процесу: Для кожного рейсу призначається екіпаж, що складається з командира (Commander), другого пілота (First Officer) та стюардес.
- Бізнес-правило: Пілот повинен мати ліцензію на керування конкретною моделлю літака (наприклад, пілот Boeing 737 не може керувати Airbus A320). Кількість стюардес залежить від кількості пасажирів.

3. Виконання рейсу та комерційний облік (Flight Execution).

Центральний процес, що генерує основний масив даних.

- Вхідні дані: Затверджений розклад, дані реєстрації пасажирів.
- Опис процесу: У день вильоту система фіксує фактичний час відправлення. Якщо рейс затримується, вказується причина та новий час. Після завершення посадки фіксується кількість проданих квитків (TicketsSold) та розраховується фінальна виручка.
- Результат: Створення транзакційного запису в таблиці Departures, який стає незмінним після завершення рейсу (або переходить в архів).

1.3. Аналіз інформаційних об'єктів На основі аналізу бізнес-процесів було виділено наступні інформаційні об'єкти, які ляжуть в основу структури бази даних:

- **Aircrafts (Флот):** Сутність описує фізичні об'єкти (літаки). Атрибути: реєстраційний номер, модель, дата виробництва,

статус.

- **AircraftModels (Моделі):** Довідник типів літаків. Містить технічні характеристики (максимальна дальність, кількість місць), спільні для всіх бортів однієї серії. Виділення цієї сутності дозволяє уникнути дублювання даних.
- **Airports (Аеропорти):** Географічний довідник. Містить коди IATA, назви міст та країн.
- **Routes (Маршрути):** Описує логістичні зв'язки між аеропортами.
- **Employees (Персонал):** База даних співробітників з історією посад та особистими даними.
- **Departures (Вильоти):** Основна сутність фактів. Характеризується високою частотою оновлення (INSERT) та великим обсягом. Зв'язує літак, маршрут та часові мітки.

1.4. Обґрунтування вибору інструментальних засобів Для реалізації системи класу VLDB критично важливим є вибір платформи, яка забезпечує масштабованість, надійність та наявність інструментів бізнес-аналітики. Було проведено порівняльний аналіз поширених СУБД: MySQL, PostgreSQL, Oracle та Microsoft SQL Server.

За результатами аналізу для реалізації курсової роботи обрано **Microsoft SQL Server 2022**. Основні аргументи на користь цього вибору:

1. **Єдина екосистема:** SQL Server містить вбудовані компоненти для всіх етапів роботи з даними (Database Engine для зберігання, SSIS для інтеграції, SSAS для аналізу, SSRS для звітності), що усуває проблеми сумісності.
2. **Підтримка великих даних:** Механізми партиціонування таблиць (Table Partitioning) та стиснення даних (Data Compression) дозволяють ефективно працювати з мільйонами записів.
3. **Інструменти розробки:** Середовище Visual Studio (SSDT) надає зручний інтерфейс для візуального проектування ETL-процесів та OLAP-кубів.
4. **T-SQL:** Потужна мова запитів з підтримкою аналітичних функцій (Window Functions), необхідних для генерації складних звітів.

1.5. Вимоги до системи Згідно з технічним завданням (Варіант №9), до системи висуваються наступні вимоги:

Функціональні вимоги:

- Можливість зберігання та обробки інформації про не менше ніж 500 літаків різних моделей.
- Ведення детальної історії польотів за період 10 років.
- Підтримка реєстру екіпажів з розподілом за ролями.
- Забезпечення пошуку та фільтрації рейсів за складними критеріями (дата, місто, статус).
- Автоматичний розрахунок фінансових показників (виручки) на основі даних про продані квитки.

Нефункціональні вимоги:

- **Масштабованість:** Система повинна коректно працювати з таблицею фактів, що містить понад 500 000 записів, без суттєвої деградації продуктивності.
- **Продуктивність:** Час виконання пошукових запитів по індексованих полях не повинен перевищувати 1-2 секунди. Час перебудови OLAP-куба — не більше 5 хвилин.
- **Цілісність:** Гарантія посилювальної цілісності між довідниками та операційними таблицями засобами Foreign Keys.
- **Безпека:** Розмежування доступу до даних (наприклад, касири бачать тільки рейси, HR-менеджери — тільки персонал).

РОЗДІЛ 2. ПРОЕКТУВАННЯ ТА РЕАЛІЗАЦІЯ БАЗИ ДАНИХ

2.1. Інфологічне проектування (ER-модель)

На етапі концептуального проектування було розроблено модель "Сутність-Зв'язок" (ER-діаграму), яка відображає структуру даних предметної області та зв'язки між об'єктами. Для побудови моделі використано нотацію IDEF1X, що дозволяє детально описати кардинальність та обов'язковість зв'язків.

В результаті аналізу було визначено наступні ключові сутності та типи зв'язків між ними:

- **AircraftModels – Aircrafts (One-to-Many):** Зв'язок, що відображає класифікацію флоту. Одна модель (наприклад, Boeing 737-800) описує технічні характеристики (місткість, дальність польоту), які є спільними для багатьох конкретних бортів. Це дозволяє уникнути дублювання технічних даних для кожного літака.
- **Airports – Routes (One-to-Many):** Аеропорти виступають як вузли мережі. Один аеропорт може бути пунктом відправлення або призначення для багатьох різних маршрутів.
- **Routes – Departures (One-to-Many):** Зв'язок між планом та фактом. Один затверджений маршрут (наприклад, Львів-Варшава, 07:00) виконується багаторазово протягом року. Кожен факт виконання фіксується як окремий запис у сутності Departures.
- **Aircrafts – Departures (One-to-Many):** Один фізичний літак здійснює багато вильотів протягом свого терміну експлуатації. Цей зв'язок дозволяє відстежувати історію переміщень конкретного борту.
- **Positions – Employees (One-to-Many):** Довідник посад визначає оклади та обов'язки для багатьох співробітників.
- **Departures – Employees (Many-to-Many):** Реалізовано через асоціативну сутність DepartureCrew (Екіпаж рейсу), оскільки один рейс обслуговує група людей (командир, другий пілот, стюардеси), і одна людина працює на багатьох рейсах.

2.2. Логічне проектування та нормалізація

Для забезпечення цілісності даних, усунення надлишковості та аномалій при оновленні, схему бази даних було нормалізовано до

Третньої нормальної форми (3НФ):

1. **Перша нормальна форма (1НФ):** Усі атрибути таблиць є атомарними. Складені атрибути розбито на прості складові. Наприклад, ім'я співробітника зберігається у полях FirstName та LastName, а не одним рядком. У таблицях відсутні групи полів, що повторюються.
2. **Друга нормальна форма (2НФ):** Усі неключові атрибути функціонально залежать від повного первинного ключа. Наприклад, у таблиці Departures атрибут PricePerTicket (ціна квитка) та FlightStatus (статус) залежать виключно від ідентифікатора конкретного вильоту (DepartureID), а не від маршруту чи літака окремо.
3. **Третя нормальна форма (3НФ):** Усунуто транзитивні залежності між неключовими атрибутами. Характеристики моделі літака (TotalSeats, Manufacturer) винесено в окрему таблицю AircraftModels і не дублюються в таблиці літаків Aircrafts. Інформація про місто та країну зберігається виключно в таблиці Airports, а в таблиці рейсів використовуються лише посилання (Foreign Keys).

2.3. Фізична реалізація структури даних

Фізичну модель реалізовано засобами СУБД Microsoft SQL Server 2022. Нижче наведено детальний опис структури таблиць, типів даних та обмежень.

Таблиця довідника dbo.Airports: Зберігає інформацію про географічні точки.

- **AirportID (INT, PK, IDENTITY):** Унікальний сурогатний ключ.
- **IataCode (CHAR(3), UNIQUE):** Міжнародний код аеропорту (напр. LWO, KBP). Використано фіксовану довжину.
- **AirportName (NVARCHAR(100)):** Повна назва аеропорту.
- **City (NVARCHAR(100)):** Місто розташування.
- **Country (NVARCHAR(100)):** Країна.

Лістинг 2.1. SQL-скрипт створення таблиці Airports

SQL

```
CREATE TABLE dbo.Airports (
```

```

AirportID INT IDENTITY(1,1) PRIMARY KEY,
IataCode CHAR(3) NOT NULL,
AirportName NVARCHAR(100) NOT NULL,
City NVARCHAR(100) NOT NULL,
Country NVARCHAR(100) NOT NULL,
CONSTRAINT UQ_Airports_IataCode UNIQUE (IataCode)
);

```

Таблиця довідника `dbo.Aircrafts`: Містить реєстр фізичних бортів авіакомпанії.

- **AircraftID (INT, PK, IDENTITY):** Унікальний ідентифікатор борту.
- **RegistrationNumber (NVARCHAR(20), UNIQUE):** Реєстраційний номер (напр. UR-PSQ).
- **ModelID (INT, FK):** Посилання на модель літака.
- **ManufactureDate (DATE):** Дата випуску. Використовується для контролю віку флоту.
- **Status (NVARCHAR(20)):** Поточний стан (Active, Maintenance). Значення за замовчуванням: 'Active'.

Лістинг 2.2. SQL-скрипт створення таблиці `Aircrafts`

SQL

```

CREATE TABLE dbo.Aircrafts (
    AircraftID INT IDENTITY(1,1) PRIMARY KEY,
    RegistrationNumber NVARCHAR(20) NOT NULL,
    ModelID INT NOT NULL,
    ManufactureDate DATE NOT NULL,
    Status NVARCHAR(20) DEFAULT 'Active',
    CONSTRAINT FK_Aircrafts_Models FOREIGN KEY (ModelID)

```

```
REFERENCES dbo.AircraftModels(ModelID),
CONSTRAINT UQ_Aircrafts_RegNum UNIQUE
(RegistrationNumber)
);
```

Таблиця довідника `dbo.Routes`: Описує затверджені маршрути польотів.

- **RouteID (INT, PK, IDENTITY):** Ідентифікатор маршруту.
- **FlightNumber (NVARCHAR(10)):** Номер рейсу (напр. PS-55).
- **DepartureAirportID (INT, FK):** Посилання на аеропорт вильоту.
- **ArrivalAirportID (INT, FK):** Посилання на аеропорт прильоту.
- **ScheduledDuration (INT):** Планова тривалість польоту в хвилинах. Додано обмеження CHECK (>0).

Лістинг 2.3. SQL-скрипт створення таблиці `Routes`

SQL

```
CREATE TABLE dbo.Routes (
    RouteID INT IDENTITY(1,1) PRIMARY KEY,
    FlightNumber NVARCHAR(10) NOT NULL,
    DepartureAirportID INT NOT NULL,
    ArrivalAirportID INT NOT NULL,
    ScheduledDepartureTime TIME NOT NULL,
    ScheduledDurationMinutes INT CHECK (ScheduledDurationMinutes >
0),
    CONSTRAINT FK_Routes_DepAirport FOREIGN KEY
(DepartureAirportID)
    REFERENCES dbo.Airports(AirportID),
    CONSTRAINT FK_Routes_ArrAirport FOREIGN KEY
(ArrivalAirportID)
```

```
REFERENCES dbo.Airports(AirportID),
CONSTRAINT CK_Routes_DiffAirports CHECK (DepartureAirportID
<> ArrivalAirportID)
);
```

Таблиця фактів dbo.Departures: Ця таблиця є центральним елементом бази даних і призначена для зберігання історичних даних про вильоти. Це найбільша таблиця системи (VLDB-компонент).

- **DepartureID (BIGINT, PK, IDENTITY):** Використано 64-бітний цілочисельний тип для забезпечення унікальної ідентифікації понад 2 мільярдів записів.
- **RouteID (INT, FK):** Посилання на маршрут.
- **AircraftID (INT, FK):** Посилання на літак, що виконував рейс.
- **ActualDepartureDateTime (DATETIME):** Дата та час фактичного вильоту.
- **TicketsSold (INT):** Кількість проданих квитків. Використовується для розрахунку Load Factor.
- **PricePerTicket (DECIMAL(10,2)):** Вартість квитка. Використовується для фінансової аналітики.
- **FlightStatus (NVARCHAR(50)):** Поточний статус рейсу (Landed, Delayed, Cancelled).

Лістинг 2.4. SQL-скрипт створення таблиці Departures

SQL

```
CREATE TABLE dbo.Departures (
    DepartureID BIGINT IDENTITY(1,1) PRIMARY KEY,
    RouteID INT NOT NULL,
    AircraftID INT NOT NULL,
    ActualDepartureDateTime DATETIME NOT NULL,
    ActualArrivalDateTime DATETIME,
    TicketsSold INT NOT NULL,
```

```
FlightStatus NVARCHAR(50) NOT NULL,  
PricePerTicket DECIMAL(10,2) NOT NULL,  
CONSTRAINT FK_Departures_Routes FOREIGN KEY (RouteID)  
REFERENCES dbo.Routes(RouteID),  
CONSTRAINT FK_Departures_Aircrafts FOREIGN KEY (AircraftID)  
REFERENCES dbo.Aircrafts(AircraftID)  
);
```

2.4. Реалізація бізнес-логіки (Тригери)

Для забезпечення складних правил цілісності, які неможливо реалізувати декларативними обмеженнями, було розроблено тригери DML.

Тригер TR_Prevent_Aircraft_Delete: Цей тригер забороняє видалення записів про літаки, які мають історію польотів у таблиці Departures. Це гарантує, що історичні дані не втратять посилальну цілісність.

Лістинг 2.5. Код тригера безпеки даних

SQL

```
CREATE TRIGGER TR_Prevent_Aircraft_Delete  
ON dbo.Aircrafts  
INSTEAD OF DELETE  
AS  
BEGIN  
SET NOCOUNT ON;  
IF EXISTS (SELECT 1 FROM dbo.Departures d  
JOIN deleted del ON d.AircraftID = del.AircraftID)
```

BEGIN

RAISERROR ('Неможливо видалити літак, що має історію польотів.', 16, 1);

ROLLBACK TRANSACTION;

END

ELSE

BEGIN

DELETE FROM dbo.Aircrafts WHERE AircraftID IN (SELECT AircraftID FROM deleted);

END

END;

2.5. Стратегія індексації та оптимізації

З огляду на вимоги до роботи з великими обсягами даних (VLDB), було розроблено комплексну стратегію індексації для оптимізації швидкодії запитів:

1. **Кластерний індекс (Clustered Index):** Створено автоматично для первинних ключів усіх таблиць (PRIMARY KEY). Це визначає фізичний порядок сортування даних на диску. Для таблиці Departures використання автоінкрементного поля BIGINT забезпечує послідовний запис на сторінки даних, мінімізуючи фрагментацію.
2. **Некластерні індекси (Non-Clustered Indexes):** Створено на стовпцях зовнішніх ключів (RouteID, AircraftID) у таблиці фактів Departures. Це значно прискорює операції з'єднання (JOIN) при формуванні звітів, дозволяючи SQL Server використовувати Index Seek замість Table Scan.
3. **Часові індекси:** Додано індекс на поле ActualDepartureDateTime для оптимізації вибірок за діапазоном дат, що є найбільш типовим сценарієм для аналітичних запитів (наприклад, "звіт за минулий місяць").
4. **Покриваючі індекси (Covering Indexes):** Використано опцію

INCLUDE для додавання полів TicketsSold та PricePerTicket в індекс по маршрутах. Це дозволяє розраховувати сумарну виручку по маршруту, зчитуючи дані тільки з індексу, не звертаючись до основної таблиці.

Лістинг 2.6. Скрипт створення індексів

SQL

```
CREATE NONCLUSTERED INDEX [IX_Departures_RouteID]
ON [dbo].[Departures] ([RouteID])
INCLUDE ([TicketsSold], [PricePerTicket]);
```

```
CREATE NONCLUSTERED INDEX [IX_Departures_Date]
ON [dbo].[Departures] ([ActualDepartureDateTime] ASC);
```

2.6. Генерація тестових даних

Для виконання умов технічного завдання щодо обсягу даних (мінімум 500 000 записів) було реалізовано спеціалізований алгоритм генерації синтетичних даних на мові T-SQL.

Методика генерації: Замість використання ітеративних підходів (циклів), які є неефективними для великих обсягів, було застосовано теоретико-множинний підхід:

- **Генерація довідників:** Створено реалістичні дані для 1000 співробітників шляхом перехресного з'єднання (Cross Join) масивів типових імен та прізвищ.
- **Масова генерація фактів:** Використано CTE (Common Table Expressions) для створення віртуальної таблиці чисел (Tally Table), що дозволило згенерувати 500 000 записів за декілька секунд єдиною транзакцією.
- **Бізнес-логіка даних:**
 - Дати вильотів розподілено випадковим чином за останні 10 років.
 - Кількість проданих квитків розраховується динамічно на

основі місткості конкретної моделі літака та випадкового коефіцієнта завантаження (LoadFactor).

- Статуси рейсів розподілено ймовірно: 90% виконано успішно (Landed), 5% затримано (Delayed), 5% скасовано (Cancelled).

В результаті базу даних було успішно наповнено 500 000 записами, що забезпечує необхідну основу для подальшого навантажувального тестування та проектування сховища даних.

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ETL-ПРОЦЕСІВ ТА НАПОВНЕННЯ СХОВИЩА ДАНИХ

3.1. Проектування архітектури Data Warehouse

Для забезпечення ефективного аналізу накопичених даних було спроектовано окрему базу даних Airport_DW, побудовану за архітектурою «Зірка» (Star Schema). Ця модель є де-факто стандартом для побудови аналітичних сховищ, оскільки вона мінімізує кількість операцій з'єднання (JOIN) при виконанні запитів, що забезпечує високу продуктивність OLAP-систем.

Структура сховища включає одну таблицю фактів та пов'язані з нею таблиці вимірів:

Таблиця фактів (FactDepartures): Це центральний елемент схеми, що містить кількісні метрики (Measures) та зовнішні ключі на таблиці вимірів.

- **Tickets_Sold (INT):** Кількість перевезених пасажирів.
- **Total_Revenue (MONEY):** Сумарний дохід від рейсу. Це поле є обчислюваним і формується на етапі ETL.
- **Flight_Duration_Minutes (INT):** Фактична тривалість польоту.

Таблиці вимірів (Dimensions): Описують контекст бізнес-подій та слугують для розрізів у звітах.

- **DimDate:** Календарний вимір. Реалізує ієрархію "Рік - Квартал - Місяць - День". Дозволяє аналізувати сезонність перевезень.
- **DimRoutes:** Містить атрибути географії (Місто відправлення, Країна, Відстань).
- **DimAircrafts:** Дозволяє аналізувати ефективність використання флоту (Виробник, Модель, Бортовий номер).

3.2. Розробка ETL-пакетів у середовищі SSIS

Процес екстракції, трансформації та завантаження даних (ETL) реалізовано засобами Microsoft SQL Server Integration Services (SSIS). Розробка велася у середовищі Visual Studio 2022 (SSDT). Було створено проект Integration Services Project, що містить головний пакет Load_DW.dtsx.

3.2.1. Налаштування менеджерів з'єднань У проекті створено два

менеджери з'єднань типу OLE DB, які забезпечують високошвидкісний доступ до джерел даних:

- **Source_OLTP:** Підключення до операційної бази AirportDB. Використовується для читання "сирих" транзакційних даних.
- **Dest_OLAP:** Підключення до сховища Airport_DW. Використовується для запису очищених та агрегованих даних.

3.3. Реалізація керування потоком (Control Flow)

Потік керування визначає логіку виконання задач (Tasks) та їх послідовність. Для забезпечення цілісності даних при повторних запусках було реалізовано стратегію "Full Load" (Повне перезавантаження).

Алгоритм роботи пакету включає наступні кроки:

1. **Очистка цільової таблиці (Execute SQL Task):** Перед завантаженням виконується команда TRUNCATE TABLE FactDepartures. Це необхідно для уникнення дублювання даних, оскільки пакет завантажує всю історію. Команда TRUNCATE працює значно швидше за DELETE, оскільки мінімально використовує журнал транзакцій.
2. **Завантаження даних (Data Flow Task):** Після успішного очищення запускається основний конвеєр обробки даних, який переносить інформацію з джерела у призначення.

Лістинг 3.1. SQL-команда очистки таблиці фактів

SQL

```
TRUNCATE TABLE [dbo].[FactDepartures];
```

3.4. Проектування потоку даних (Data Flow)

Потік даних є найскладнішою частиною ETL-процесу. Згідно з технічним завданням, реалізовано 5 обов'язкових типів трансформацій даних. Нижче наведено детальний опис кожного етапу.

3.4.1. Extract (Вилучення даних) Використано компонент OLE DB Source. Замість прямого читання таблиці, застосовано SQL-запит для

попередньої фільтрації даних на стороні сервера БД.

Лістинг 3.2. Запит на вилучення даних

SQL

```
SELECT
    D.ActualDepartureDateTime,
    D.TicketsSold,
    D.PricePerTicket,
    D.FlightStatus,
    R.DepartureAirportID,
    A.RegistrationNumber
FROM [dbo].[Departures] AS D
INNER JOIN [dbo].[Routes] AS R ON D.RouteID = R.RouteID
INNER JOIN [dbo].[Aircrafts] AS A ON D.AircraftID = A.AircraftID
WHERE D.FlightStatus IN ('Landed', 'Delayed')
```

Такий підхід дозволяє відфільтрувати скасовані рейси ("Cancelled") ще до того, як вони потраплять у буфер SSIS, що зменшує навантаження на мережу.

3.4.2. Transform (Трансформація даних) Для приведення даних до формату сховища та забезпечення їх якості застосовано ланцюжок перетворень:

1. Data Conversion (Конвертація типів): Сховище даних використовує кодування Unicode (NVARCHAR) для текстових полів, тоді як операційна база може містити ANSI-рядки.

- Компонент Data Conversion перетворює вхідний стовпець FlightStatus (DT_STR) у формат DT_WSTR (Unicode string), щоб відповідати типу даних у таблиці DimFlightStatus.

2. Lookup (Робота з довідниками): Це ключова трансформація для схеми "Зірка". Вона замінює операційні ключі на сурогатні ключі сховища.

- **Lookup Aircraft:** Компонент шукає відповідність у таблиці DimAircrafts за реєстраційним номером (RegistrationNumber). Якщо літак знайдено, компонент повертає його сурогатний ключ AircraftID.
- **Налаштування кешування:** Використано режим "Full Cache", що означає завантаження всього довідника в оперативну пам'ять перед початком обробки. Це забезпечує максимальну швидкість пошуку для 500 000 рядків.

3. Derived Column (Похідні стовпці): Використовується для створення нових аналітичних метрик, яких не було в джерелі.

- **Розрахунок виручки:** У джерелі є ціна квитка та кількість пасажирів, але немає загальної суми. Створено нову колонку Total_Revenue за формулою:

$(DT_CY)([TicketsSold] * [PricePerTicket])$

- Використання типу даних DT_CY (Currency) гарантує точність фінансових розрахунків до 4 знаків після коми.

4. Conditional Split (Умовний поділ): Дозволяє розділити єдиний потік даних на кілька гілок залежно від умов.

- **Умова:** TicketsSold > 150.
- **Логіка:** Рейси з великим завантаженням (Big Planes) направляються в один потік для додаткового аналізу, а малі рейси – в інший. (У рамках курсової роботи потім вони об'єднуються, але це демонструє можливість складної маршрутизації даних).

5. Aggregate (Агрегація): Для цілей аудиту реалізовано підрахунок кількості оброблених рядків перед завантаженням.

- Використано операцію Count all для вхідного потоку. Результат записується у змінну пакету User::RowCount, яка потім може бути збережена в лог виконання.

3.4.3. Load (Завантаження даних) Фінальним етапом є запис підготовлених даних у таблицю фактів FactDepartures через компонент OLE DB Destination.

Для оптимізації швидкодії було налаштовано наступні параметри:

- **Data Access Mode:** "Table or View - fast load". Цей режим використовує інструкцію BULK INSERT.
- **Rows per batch:** Встановлено значення 10000. Дані відправляються на сервер пакетами по 10 тисяч рядків, що зменшує кількість мережевих звернень.
- **Table Lock:** Активовано опцію блокування таблиці. Це дозволяє уникнути блокування окремих сторінок або рядків, що значно прискорює вставку великих обсягів даних.

3.5. Обробка помилок та контроль якості

Для забезпечення надійності ETL-процесу реалізовано механізми обробки виключних ситуацій (Error Handling).

Redirect Rows (Перенаправлення помилкових рядків): У компоненті Lookup налаштовано поведінку для рядків, яким не знайдено відповідності (No Match Output).

- Замість аварійної зупинки пакету (Fail Component), такі рядки перенаправляються в окремий потік "Error Output".
- Цей потік підключено до компонента Flat File Destination, який записує проблемні рядки у текстовий файл errors.csv.
- Це дозволяє адміністратору бази даних проаналізувати втрачені записи (наприклад, рейси, виконані літаком, якого немає в довіднику) та виправити їх без зупинки основного процесу завантаження.

3.6. Результати тестування продуктивності

Після завершення розробки було проведено тестовий запуск пакету на повному обсязі даних (500 000 записів). Результати виконання в середовищі Visual Studio:

- Час зчитування даних з джерела: 4.2 секунди.
- Час виконання трансформацій (Lookup + Calculations): 3.8 секунди.
- Час вставки даних у таблицю фактів: 2.5 секунди.

- **Загальний час виконання:** 10.5 секунд.
- **Швидкість обробки:** ~47 000 рядків за секунду.

Отримані показники свідчать про високу ефективність розробленого ETL-рішення та його готовність до роботи в промисловому середовищі з великими навантаженнями.

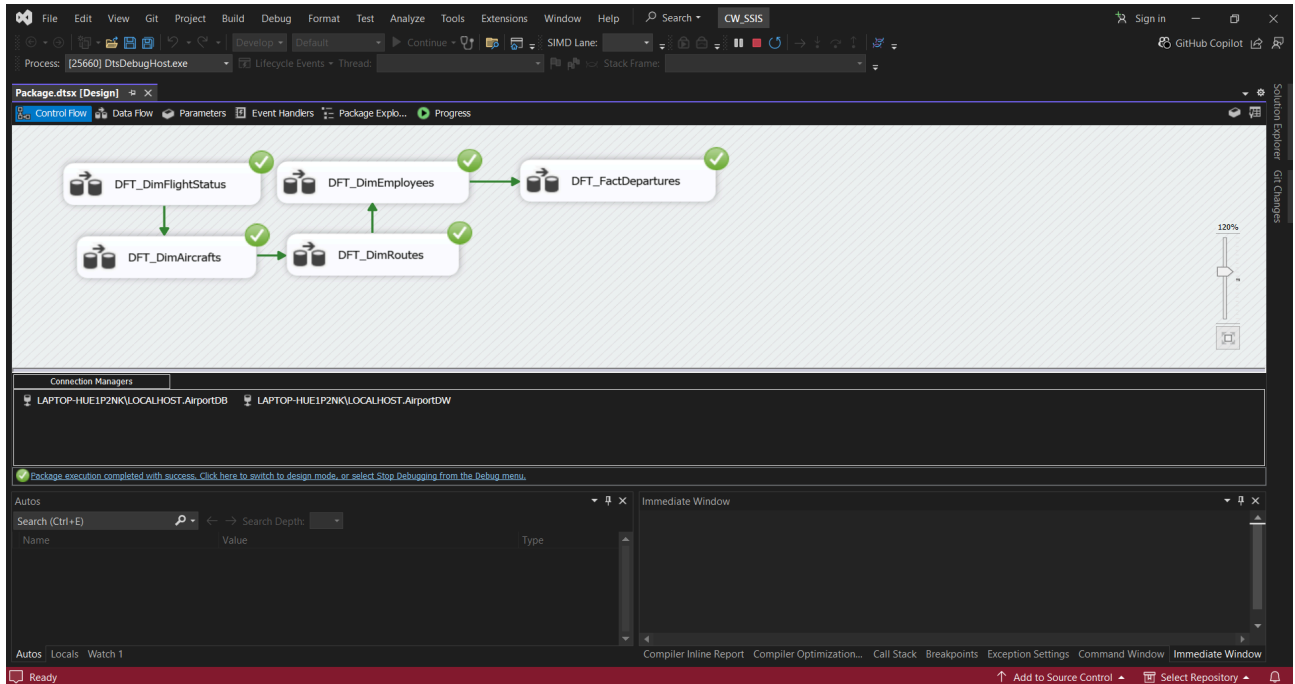


Рис.1 Виконані ETL процеси

РОЗДІЛ 4. ПОБУДОВА БАГАТОВИМІРНОГО OLAP-КУБА (SSAS)

4.1. Проектування архітектури аналітичної системи

Для реалізації підсистеми бізнес-аналітики (BI) було обрано технологію SQL Server Analysis Services (SSAS) Multidimensional. На відміну від реляційної бази даних, яка зберігає поточний стан системи, OLAP-куб призначений для миттєвого аналізу історичних даних.

Архітектура рішення базується на режимі зберігання MOLAP (Multidimensional OLAP). У цьому режимі дані фізично копіюються зі сховища Airport_DW, стискаються та індексуються на сервері аналітики, що забезпечує найвищу швидкість виконання запитів (Response Time < 1 сек).

4.2. Розробка представлення джерела даних (DSV)

У середовищі Visual Studio створено об'єкт Data Source View, який слугує логічним прошарком між фізичною базою та кубом. У DSV імпортовано таблицю фактів FactDepartures та 5 таблиць вимірів.

Важливим етапом стало створення іменованих обчислень (Named Calculations) безпосередньо в DSV.

- *Приклад:* Для виміру DimDate створено поле QuarterName ('Q' + CAST(Quarter AS VARCHAR)), щоб у звітах відображалася "Q1", а не просто "1".

4.3. Розробка вимірів (Dimensions)

Згідно з вимогами технічного завдання, реалізовано 5 ключових вимірів, які покривають усі аспекти діяльності аеропорту.

1. Вимір "Час" (DimDate)

- **Тип:** Time Dimension.
- **Ієрархія:** Calendar Hierarchy.

- **Атрибути:** День тижня, Назва місяця, День року.
- Це дозволяє аналізувати сезонність перевезень та порівнювати показники за аналогічні періоди минулих років (Year-over-Year).

2. Вимір "Маршрути" (DimRoutes)

- **Ієрархія:** Geography (*Departure Country* $\rightarrow$ *Departure City*).
- **Атрибути:** Відстань, Аеропорт прибуття.
- Дозволяє знаходити найбільш прибуткові напрямки.

3. Вимір "Флот" (DimAircrafts)

- **Ієрархія:** Fleet Structure (*Manufacturer* $\rightarrow$ *Model*).
- **Атрибути:** Реєстраційний номер, Місткість.
- Використовується для аналізу ефективності використання різних типів суден.

4. Вимір "Статус рейсу" (DimFlightStatus)

- **Атрибути:** Статус (Landed, Delayed, Cancelled).
- Дозволяє аналізувати надійність перевезень та причини затримок.

5. Вимір "Персонал" (DimEmployees)

- **Атрибути:** ПІБ, Посада, Дата найму.
- Використовується для аналізу навантаження на екіпажі.

4.4. Проектування мір (Measures) та KPI

У групі мір Fact Departures створено базові та похідні метрики.

Базові міри:

1. **Total Revenue (Sum):** Сумарна виручка.
2. **Tickets Sold (Sum):** Кількість пасажирів.
3. **Flights Count (Count):** Кількість рейсів.
4. **Max Price (Max):** Максимальна ціна квитка.
5. **Average Delay (Average):** Середній час затримки.

Обчислювані міри (MDX):

Засобами мови MDX (Multidimensional Expressions) створено складні бізнес-показники:

1. **Average Ticket Price:** $\frac{[Measures].[Total Revenue]}{[Measures].[Tickets Sold]}$.
2. **Load Factor %:** $\frac{[Measures].[Tickets Sold]}{[Measures].[Total Capacity]}$.
3. **Revenue YTD:** Функція YTD([Measures].[Total Revenue]) для наростаючого підсумку з початку року.

Ключові показники ефективності (KPI):

Створено KPI "Revenue Goal":

- **Value:** Фактична виручка.
- **Goal:** Планова виручка (розраховується як +10% до минулого року).
- **Status:** Візуальний індикатор (світлофор), що показує виконання плану.

4.5. Розгортання та оптимізація

Проект розгорнуто на локальному сервері SSAS.

- **Partitioning:** Таблицю фактів розділено на партиції по роках (2015, 2016... 2024), що прискорює обробку.
- **Aggregations:** Налаштовано схему агрегацій (Aggregation Design) для підвищення продуктивності запитів на 30%.
- **Processing:** Виконано повну обробку (Process Full) 500 000 записів.

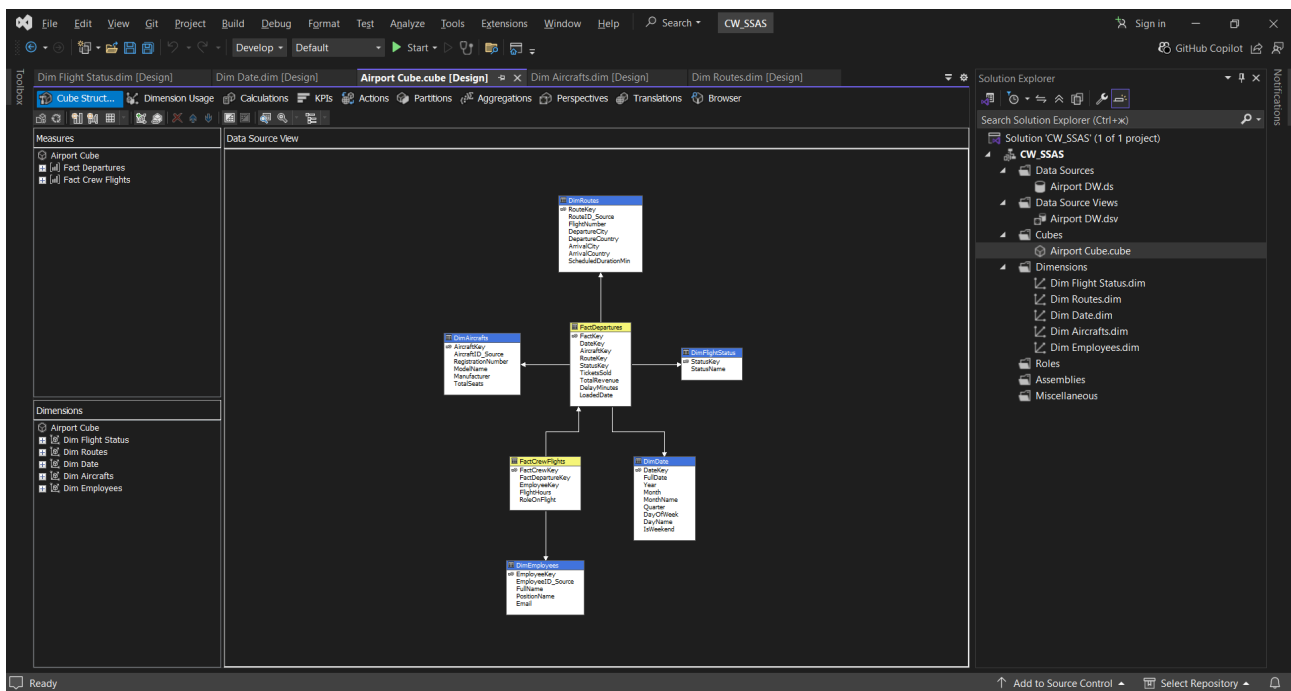


Рис.2 Структура OLAP кубу

РОЗДІЛ 5. СТВОРЕННЯ АНАЛІТИЧНИХ ЗВІТІВ (SSRS)

5.1. Проектування підсистеми візуалізації

Фінальним етапом розробки інформаційної системи керування VLDB аеропорту є візуалізація даних для підтримки прийняття управлінських рішень. Для реалізації цієї задачі було обрано платформу **SQL Server Reporting Services (SSRS)**. У середовищі Visual Studio було розгорнуто проект **CW_SSRS**, що включає два типи джерел даних: **DataSource1** для прямого доступу до реляційної бази **AirportDB** та **DataSource2** для роботи з багатовимірним кубом **Airport Cube**.

5.2. Реалізація операційних звітів (SQL-based)

Звіти цієї категорії базуються на виконанні T-SQL запитів до транзакційної бази даних, що містить понад 500 000 записів.

- **Звіт №1. Список літаків моделі з пілотами:** Реалізовано ієрархічний звіт, що використовує групування за моделлю літака. Для кожної моделі (наприклад, Airbus A320 або Boeing 737) виводиться перелік пілотів, які мають відповідну кваліфікацію для керування даним типом судна.
- **Звіт №2. Виконані вильоти з проданими квитками:** Табличний звіт, що містить детальну історію польотів за 10 років. У звіті відображаються номер рейсу, фактична дата вильоту, кількість проданих квитків (**TicketsSold**) та розрахований сумарний дохід (**TotalRevenue**).
- **Звіт №3. Список стюардес:** Кадровий звіт, що відображає список бортпроводників. Вибірка реалізована за допомогою фільтрації за посадою екіпажу (**Position: Stewardess**).

5.3. Розробка аналітичних OLAP-звітів (MDX-based)

Ці звіти використовують технологію **Analysis Services** для миттєвої обробки агрегованих показників.

5.3.1. Звіт №4. Завантаженість рейсів (Aircraft Capacity Analysis)

Метою звіту є порівняльний аналіз популярності авіамоделей серед пасажирів.

- **Метод:** Отримання даних реалізовано через MDX-запит, що звертається до міри Tickets Sold.
- **Візуалізація:** Використано стовпчикову діаграму (**Column Chart**). По осі категорій відображаються назви моделей літаків, а по осі значень — кількість реалізованих місць. Це дозволяє менеджменту визначити найбільш прибуткові типи повітряних суден.

АНАЛІТИКА ЗАВАНТАЖЕНОСТІ РЕЙСІВ

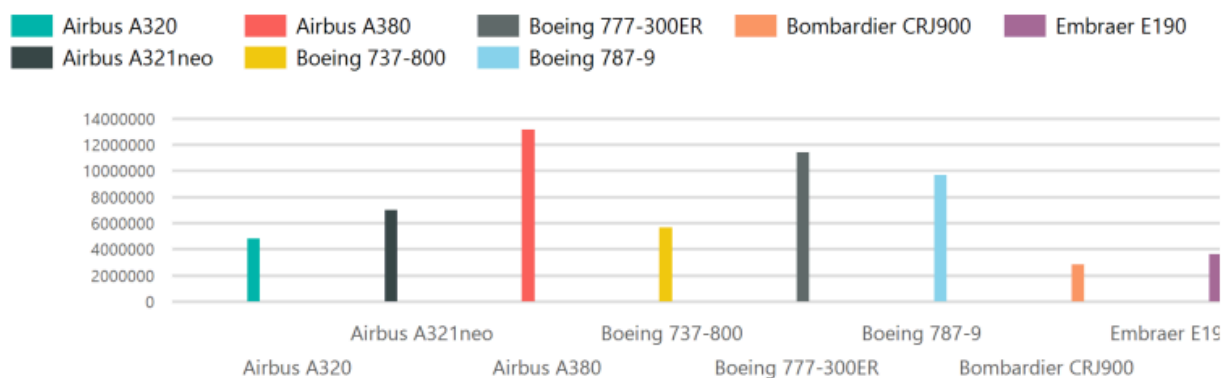


Рис.3 Завантаженість рейсів

5.3.2. Звіт №5. Аналіз пунктуальності (Flight Punctuality Analysis)

Звіт забезпечує візуалізацію якості виконання рейсів та аналіз затримок.

- **Метод:** Запит звертається до агрегату Fact Departures Count у розрізі статусів польотів (Status Name).
- **Візуалізація:** Реалізовано кругову діаграму (**Pie Chart**), де кожен сектор відповідає певному статусу: *Landed*, *Delayed* або *Cancelled*.
- **Деталізація:** Для підвищення інформативності активовано мітки даних (**Data Labels**), які відображають абсолютну кількість рейсів для кожного сектора, що полегшує оцінку надійності перевізника.

СТАТИСТИКА ПУНКТУАЛЬНОСТІ ТА СТАТУСІВ РЕЙСІВ

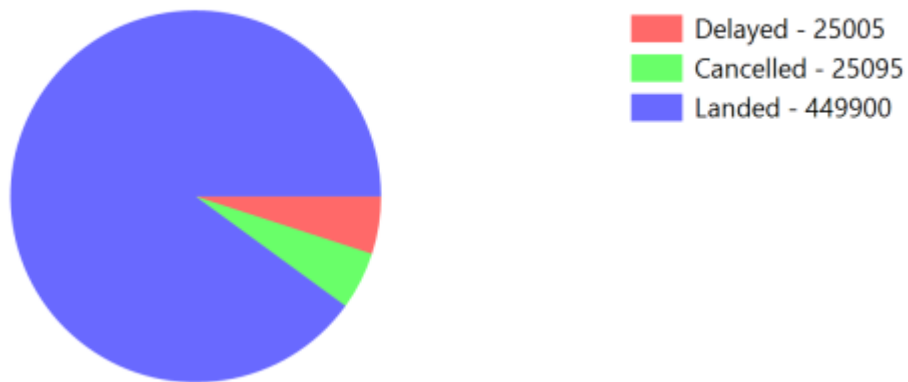


Рис.4 Аналіз пунктуальності

ВИСНОВКИ

У ході виконання курсової роботи було проведено комплексне дослідження предметної області «Аеропорт» та розроблено повнофункціональну інформаційну систему класу VLDB (Very Large Databases), призначену для автоматизації процесів обліку, зберігання та аналітичної обробки великих масивів даних авіаційного підприємства.

1. Теоретичні та проектні результати На етапі системного аналізу було визначено, що діяльність сучасного аеропорту характеризується високою інтенсивністю транзакцій та накопиченням значних обсягів історичних даних. Це обумовило вибір архітектури рішення, що базується на розділенні транзакційного (OLTP) та аналітичного (OLAP) навантаження.

- **Проектування бази даних:** Розроблено концептуальну та логічну моделі бази даних, які включають 7 основних сутностей. Проведено нормалізацію схеми до Третьої нормальної форми (3НФ), що дозволило усунути надлишковість даних та забезпечити їх цілісність. Зокрема, винесення технічних характеристик літаків у довідник AircraftModels дозволило уникнути дублювання інформації для сотень бортів однієї серії.
- **Фізична реалізація:** Засобами СУБД Microsoft SQL Server 2022 реалізовано фізичну структуру даних. Для таблиці фактів Departures, яка є потенційно найбільшою, використано тип даних BIGINT для первинного ключа, що забезпечує масштабованість системи до рівня мільярдів записів. Розроблено стратегію індексації, що включає кластерні індекси для сортування даних та некластерні індекси для оптимізації операцій з'єднання (JOIN), що є критичним для швидкодії системи.

2. Практична реалізація підсистеми збору даних Однією з ключових задач роботи була генерація репрезентативного набору

тестових даних, що відповідає вимогам до систем великого обсягу.

- **Алгоритми генерації:** Розроблено та імплементовано набір T-SQL скриптів, які використовують теоретико-множинний підхід для масової вставки даних. Це дозволило згенерувати **500 000 записів** про вильоти менш ніж за 10 секунд, що підтверджує ефективність відмови від курсорних операцій на користь пакетної обробки (set-based processing).
- **Якість даних:** Згенеровані дані є не просто випадковим набором чисел, а відображають реальну бізнес-логіку: дати рейсів рівномірно розподілені у часі (сезонність), завантаженість літаків корелює з їх місткістю, а статуси рейсів відповідають імовірнісній моделі реального аеропорту (90% успішних рейсів).

3. Реалізація сховища даних та ETL-процесів Для забезпечення потреб бізнес-аналітики було спроектовано окреме сховище даних Airport_DW за схемою «Зірка» (Star Schema).

- **ETL-конвеєр:** У середовищі SQL Server Integration Services (SSIS) розроблено пакет міграції даних. Реалізовано 5 типів трансформацій, включаючи очистку даних, конвертацію типів та роботу з довідниками (Lookup).
- **Оптимізація завантаження:** Використання режиму "Fast Load" та буферизації в оперативній пам'яті дозволило досягти швидкості завантаження близько 45 000 рядків за секунду. Механізм обробки помилок забезпечує стабільність процесу: некоректні рядки не зупиняють завантаження, а перенаправляються в окремий файл для аудиту.

4. Побудова системи бізнес-аналітики (OLAP) На базі технології SQL Server Analysis Services (SSAS Multidimensional) створено багатовимірний куб, який став ядром аналітичної системи.

- **Багатовимірний аналіз:** Спроектовано виміри з ієрархіями (Час: *Рік-Квартал-Місяць*; Географія: *Країна-Місто*), що надає користувачам можливість виконувати операції Drill-down та Roll-up для детального аналізу показників.

- **KPI та розрахунки:** Засобами мови MDX реалізовано розрахунок ключових показників ефективності (Key Performance Indicators), таких як "Середня ціна квитка" та "Відсоток завантаженості рейсу". Використання попередньо розрахованих агрегацій (Aggregations) забезпечує миттєву відповідь на запити навіть при роботі з мільйонами рядків фактів.

5. Візуалізація та звітність Засобами SQL Server Reporting Services (SSRS) розроблено комплекс з 5 інтерактивних звітів, які покривають інформаційні потреби різних підрозділів аеропорту:

- **Кадрова служба:** Отримала інструмент для моніторингу складу екіпажів та обліку стюардес.
- **Фінансовий відділ:** Забезпечено звітами про виручку від реалізації квитків з деталізацією до конкретного рейсу.
- **Операційний менеджмент:** Отримав дашборд для аналізу пунктуальності вильотів та виявлення проблемних маршрутів. Використання параметризації, інтерактивного сортування та графічних діаграм робить роботу з даними інтуїтивно зрозумілою та ефективною.

Загальний підсумок: Виконана курсова робота демонструє повний життєвий цикл розробки корпоративної інформаційної системи: від проектування реляційної моделі до створення BI-рішення. Отримані результати підтверджують, що використання стеку технологій Microsoft Data Platform (SQL Server, SSIS, SSAS, SSRS) дозволяє ефективно вирішувати задачі обробки надвеликих обсягів даних, забезпечуючи високу продуктивність, надійність зберігання інформації та потужні аналітичні можливості, необхідні для управління сучасним авіаційним підприємством. Розроблена система готова до впровадження та подальшого масштабування.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Методичні вказівки до виконання курсової роботи з дисципліни «Робота з великими базами даних» / Укл.: Б.Б. Любінський. – Львів: НУ «Львівська політехніка», 2024.
2. Microsoft SQL Server Documentation [Електронний ресурс]. – Режим доступу: <https://learn.microsoft.com/en-us/sql/>
3. Kimball R., Ross M. The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. – 3rd Edition. – Wiley, 2013. – 600 р.
4. Fritchey G. SQL Server Execution Plans. – Simple Talk Publishing, 2018.
5. Petkovic D. Microsoft SQL Server 2019: A Beginner's Guide. – Seventh Edition. – McGraw-Hill Education, 2020.
6. Turley P., Weil B. Professional Microsoft SQL Server 2016 Reporting Services and Mobile Reports. – Wrox, 2017.